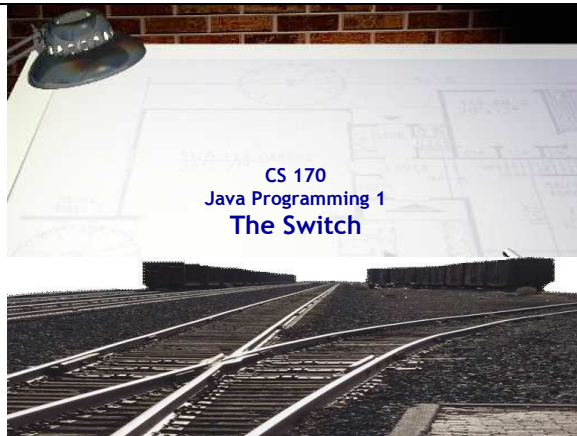




CS 170
Java Programming 1
The Switch

Slide 1

CS 170

Java Programming 1

The Switch

Duration: 00:00:46

Advance mode: Auto

Notes:

Hello. Welcome to the CS 170, Java Programming 1 lecture on the switch statement. As you can see from the last section, nested if statements, as well as the ladder-style if-else-if statements, provide you with the tools you need to do multi-way branches.

Another Java control structure, the switch statement, provides an even more efficient way to select between several different alternatives, provided you can live with its limitations and somewhat idiosyncratic behavior.

At the end of this lecture, we'll also look at Java's selection operator. Let's start, though, by looking at the situations where a switch statement is the appropriate choice.

Menu-Style Code

- With ladder-style **if-else-if**, you might sometimes find yourself writing "menu-style" code that looks like this:

```
int choice = getUserChoice();
if ( choice == 1 ) ...
else if (choice == 2) ...
else if (choice == 3) ...
else ...
```

 - Notice that each condition is testing the same variable
 - Each test is made against a literal expression
- Java has a **more efficient** way to do this: the **switch**

Slide 2

Menu-Style Code

Duration: 00:01:00

Advance mode: Auto

Notes:

Suppose you needed to write the code to run a vending machine or handle a drop-down menu. You might find yourself writing code that looks something like this.

In the fragment shown here, we call a function to get the user's selection and store it in an int variable named choice. The code then uses the multiple-selection pattern that you learned about in the previous lesson to carry out an appropriate action depending on the item selected by the user, whether that's dispensing Twinkies or Tofu, or starting up the Oracle server.

Notice that in this case, we only have one integer input variable and that each test condition compares that variable, using the equality operator, against a set of literal values.

When both of these conditions are true, using Java's switch statement instead of a chain of sequential ifs, is more efficient, and probably a little clearer as well.

What is the switch?

- A selection statement based on **integer evaluation**
 - Named after the switches used by the phone company
 - Dial a number and get connected to your party
 - Think also, of a menu or vending machine

Notes:

The switch statement is a multi-way selection structure based on the implicit comparison of an integer selector to a collection of labeled blocks of code. It doesn't use an explicit boolean condition like other selection statements do.

Many other languages have a multi-way branch statement, which is usually called a case statement. If you've programmed in one of these different languages, you might find the term switch a little unusual. The name seems much more logical, however, if you think of a railway switching yard (like the picture on the opening slide). The switch statement

Slide 3

What is the switch?

Duration: 00:01:30

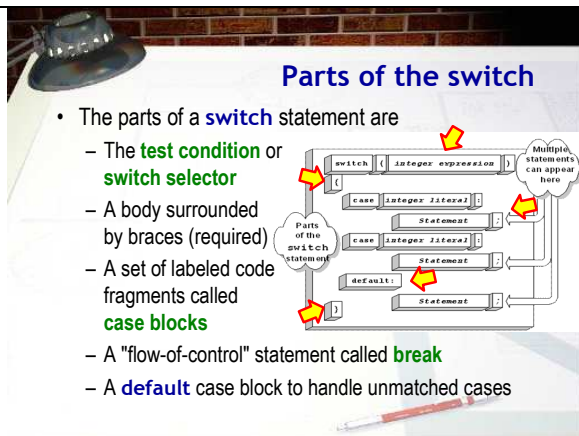
Advance mode: Auto

was actually named after the telephone switch, like this. The switch statement is one of the syntax features that Java inherited from the C programming language, which was invented at Bell Labs when it was part of the national AT&T telephone network.

With a telephone switch, you supply a phone number and the switch connects you with a particular subscriber. With Java's switch, you supply a "code block" number, and you're immediately routed to that particular block of numbered code.

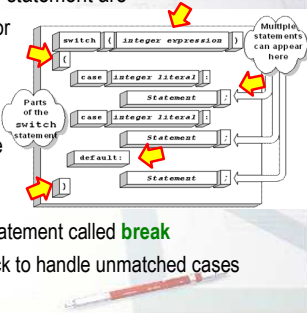
Menus and vending machines also work in a similar manner. You press the specific code on the vending machine, D2, for instance, and out pops a Snickers bar.

Let's briefly look at the syntax of the switch statement, then we'll take a look at how it works.



Parts of the switch

- The parts of a **switch** statement are
 - The **test condition** or **switch selector**
 - A body surrounded by braces (required)
 - A set of labeled code fragments called **case blocks**
 - A "flow-of-control" statement called **break**
 - A **default** case block to handle unmatched cases



Slide 4

Parts of the switch

Duration: 00:01:29

Advance mode: Auto

Notes:

Here's an illustration that shows the syntax of the switch statement. As you can see it consists of five different parts.

- First there's the switch keyword, followed by an integer expression enclosed in parentheses. This is the value that will be implicitly matched against the numbered blocks of code that you'll provide. This integer expression is called the switch selector.
- Second is the body of the switch statement, which follows the selector and is enclosed in braces. While you can have an if statement without braces, you always need them for the switch statement.
- Third are any number of code fragments contained inside the switch body. These are called case blocks. Each block of code is preceded by a case label that ends with a colon, not a semicolon. The individual statements contained in each case block do not have to be enclosed in any additional braces.
- Fourth are the break statements used to end each case. The break statement is a specific flow of control statement called a jump and it's used to separate individual code blocks.
- Lastly is a default: case block which is used to handle selectors that don't match any case. Kind of the "else" label for the switch statement.

Let's look at each of these pieces to see how they work together.

The switch Selector

- The **switch selector** can be most integer expressions
 - Includes **int**, **short**, **byte**, and **char**
 - Can't** use **boolean**, **long**, **float**, **double**, or **String**
 - Placed in parentheses following keyword **switch**

```

Using the switch Statement
switch (ch)
{
    case '1':
        lbl.setText("Here's TOFU");
        break;
    case '2':
        lbl.setText("Here's TWINKIE");
        break;
    default:
        lbl.setText("Unknown Choice");
}
  
```

Slide 5

The switch Selector

Duration: 00:00:42

Advance mode: Auto

Notes:

The first thing we want to look at is the switch selector. As I already mentioned, the selector has to be an integer expression. You can use int, short, byte or even char variables. You can't use boolean, float, double or String values. You also can't use long, even though it's an integer type. You can use float or double variables if you cast them to int, of course. In the example shown here I've used a char variable named ch.

You'll place your integer variable or expression in parentheses, following the keyword switch and before the braces that surround the switch body.

The Case Label

- Each **case block** begins with the keyword **case**
 - Followed by an **integer constant** and a **colon**
 - Called the **case label**
 - Literals or **static final** constants; no variables, ranges

```

Using the switch Statement
switch (ch)
{
    case '1':
        lbl.setText("Here's TOFU");
        break;
    case '2':
        lbl.setText("Here's TWINKIE");
        break;
    default:
        lbl.setText("Unknown Choice");
}
  
```

Slide 6

The Case Label

Duration: 00:01:23

Advance mode: Auto

Notes:

The syntax for each of the labeled blocks of code can be a little confusing. Here are the rules.

First, each block of code begins with the keyword case (all in lowercase, of course). The case keyword is followed by an integer constant and a colon (not a semi-colon). This line is called the case label.

Now, when I say that the keyword case is followed by an integer constant, I'm speaking generically. Make sure that the actual type of the literal matches the type of the selector. In my case, since I've used a char variable for the selector, the literals I've used in each case label are char literals (with the single quotes around them). If my selector variable was a short or int, though, I'd use a literal int value 1 and 2, without the single quotes.

In addition to literals, you can also use static final constants and enumerated values (which we'll learn about later in the semester). You can't use regular variables, though or expressions. There is also no way to provide a range of values using a single case label, although you can get the same effect in some situations by taking advantage of the switch statement's fall-through feature (or design flaw, depending on your point of view), which we'll cover next.

How the switch Works

- When a **case** label matches the **switch** selector
 - Java "jumps" to the code immediately following label
- Continues executing **all** the remaining code
 - This is called "**fall through**"
 - Usually put a **break** statement at end of every **case**

Using the switch Statement

```
switch (ch)
{
    case '1':
        lbl.setText("Here's TOFU");
        break;
    case '2':
        lbl.setText("Here's TWINKIE");
        break;
    default:
        lbl.setText("Unknown Choice");
}
```

Slide 7

How the switch Works

Duration: 00:00:54

Advance mode: Auto

Notes:

When a switch statement is encountered, the first thing that happens is that its integer "selector" expression is evaluated. After the selector is evaluated the following steps are performed:

1. The list of labeled constants is scanned, looking for a match. If an exact match is found then Java begins executing the first line of code immediately following the matching case label.

Each line of code following the case label is executed sequentially until either a break statement is encountered or all of the statements in the switch body have been executed (not just the statements in the selected block of code). This behavior is called "fall through", and for this reason you'll normally end each case block with a break statement, as I've done in the example here.

The default Case

- If the **switch** selector doesn't match any **case**?
 - Jumps to code following **default case** label
 - Use keyword **default** followed by a colon
 - Jump to code past **switch** if no **default** provided

Using the switch Statement

```
switch (ch)
{
    case '1':
        lbl.setText("Here's TOFU");
        break;
    case '2':
        lbl.setText("Here's TWINKIE");
        break;
    default:
        lbl.setText("Unknown Choice");
}
```

Slide 8

The default Case

Duration: 00:00:24

Advance mode: Auto

Notes:

If no case label is found to match the selector, then execution jumps to the statement following the default: label, if you've added one, as I've shown in the picture here. Since the default label isn't required, if no match is found and there is no default: label, then execution jumps to the first statement following the switch body.

Express Yourself

- Exercise 1:** What is the value of **alpha** at the end of this **switch** statement if the user enters:

- A) 1
- B) 2
- C) 3
- D) 4

```
int alpha = scanner.nextInt();
switch (alpha)
{
    case 1:
        alpha = alpha + 2;
        break;
    case 4:
        alpha++;
    case 5:
        alpha = 2 * alpha;
    case 6:
        alpha = alpha + 5;
        break;
    default:
        alpha--;
}
```

Slide 9

Express Yourself

Duration: 00:01:07

Advance mode: Auto

Notes:

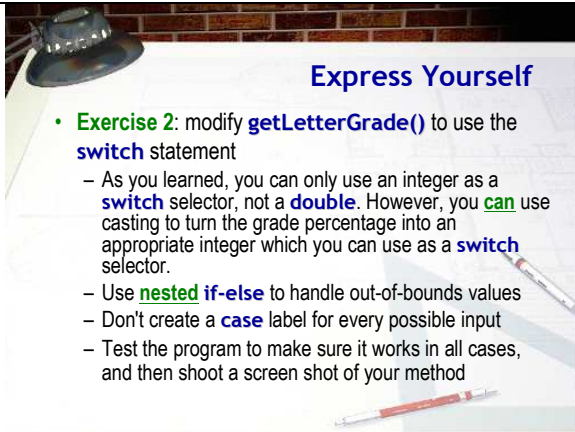
Let's look at some examples using switch statements to make sure you understand how they work. Then, you'll get an opportunity to try one for yourself. Create a new section in your in-class exercise documents and then just type in the answers. Try to figure out what the results should be without using your compiler if possible. Once you're done, you can check your answers using your compiler.

Here's a section of code using a switch statement. The variable scanner is a scanner object. Tell me what the value of alpha is at the end of this switch statement if the user enters:

- A) 1
- B) 2
- C) 3

D) 4

Notice that case 1 and case 2 use fall-through to share code between two different input values. This is how you do a range of values with the switch statement. You still have to list every possible value in the range though.



Express Yourself

- **Exercise 2:** modify `getLetterGrade()` to use the **switch** statement
 - As you learned, you can only use an integer as a **switch** selector, not a **double**. However, you **can** use casting to turn the grade percentage into an appropriate integer which you can use as a **switch** selector.
 - Use **nested if-else** to handle out-of-bounds values
 - Don't create a **case** label for every possible input
 - Test the program to make sure it works in all cases, and then shoot a screen shot of your method

Slide 10
Express Yourself
Duration: 00:01:30
Advance mode: Auto

Notes:

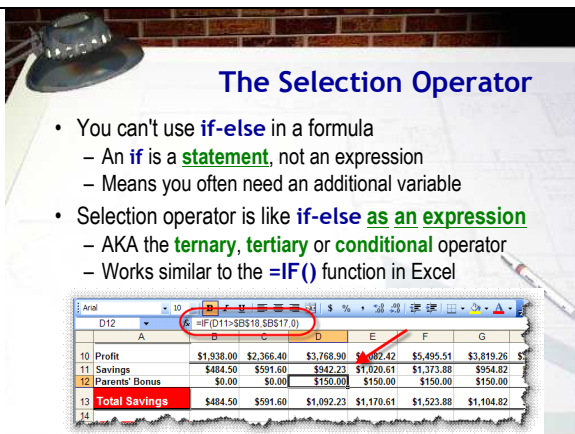
Now, let's return to the Student class and the `letterGrade()` method. Comment-out the existing code and try your hand at calculating letter grades using a switch statement.

As you've just learned, you can only use an integer as a switch selector, not a double, while the `getPercentage()` method in the student class returns a double. You can still use a switch statement to select the appropriate grade, though, by the judicious use of casting along with a little arithmetic.

There are two secrets to solving this problem in the most efficient way.

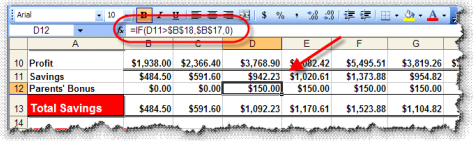
- *First, you'll probably want to use an if statement for the out-of-bounds values, since there are an infinite number of them. Then, nest your switch statement for the six discreet grade values.
- *Second, before you blindly create 102 case labels though, take a minute to think things through. You'll only need six case blocks (one for each grade including the default). How close can you come to doing that with six case labels. (You'll actually probably need at least one more).

When you're done, run your unit tests to make sure all of them still pass. Once they pass, shoot me a screen-shot of your method.



The Selection Operator

- You can't use **if-else** in a formula
 - An **if** is a **statement**, not an expression
 - Means you often need an additional variable
- Selection operator is like **if-else as an expression**
 - AKA the **ternary**, **tertiary** or **conditional** operator
 - Works similar to the **=IF()** function in Excel



Slide 11

Notes:

One problem with using if-else is that you can't use an if-else in a formula or expression; if-else is a statement; it does not produce a value. That means, you often must create an additional variable when a portion of an expression depends upon a certain condition.

Java's, selection operator provides a way to use an if-else condition inside an expression. The selection operator is sometime called the ternary, or tertiary operator, because it is the only operator to require three operands. It's also known as the conditional operator.

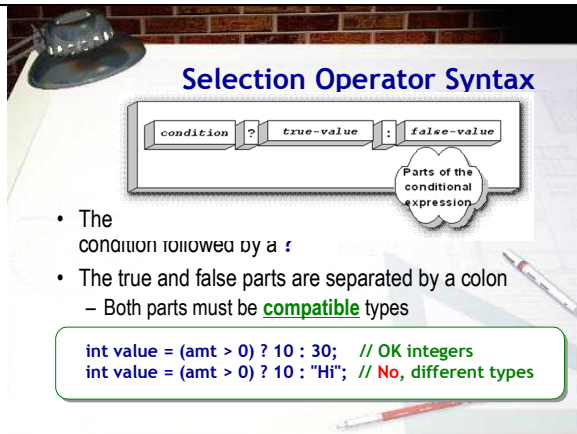
Most of you are probably familiar with Microsoft Excel. The selection operator works very much like the `IF()` function in

The Selection Operator

Duration: 00:01:34

Advance mode: Auto

Excel. In the example here, from CS 111, Harry and Matilda are trying to save up some money for a tractor. If they manage to save at least \$750 each month, their parents will match it with an additional \$150. This IF function formula first checks to see if the value in cell D11 (which is the amount saved) is greater than the bonus cutoff value stored in cell B18 by using a relational operator, just like you'd do with an if statement in Java. If the boolean condition is true, then the bonus value from cell B17 is stored in the cell; if the boolean condition is false, then the value 0 is stored in the cell.



Selection Operator Syntax

condition ? true-value : false-value

Parts of the conditional expression

- The condition followed by a `?`
- The true and false parts are separated by a colon
 - Both parts must be **compatible** types

```
int value = (amt > 0) ? 10 : 30; // OK integers
int value = (amt > 0) ? 10 : "Hi"; // No, different types
```

Slide 12

Selection Operator Syntax

Duration: 00:01:33

Advance mode: Auto

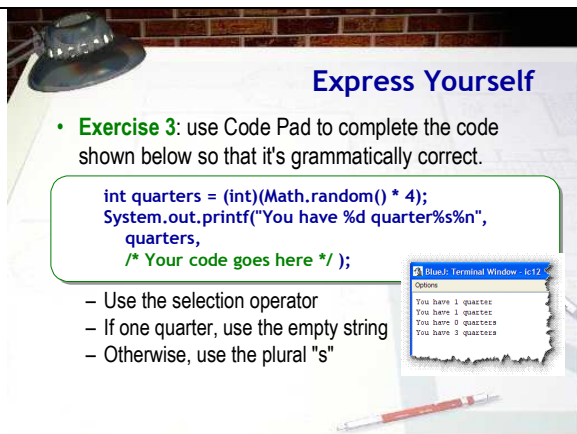
Notes:

Let's look at the syntax of the selection operator.

The first operand is the condition or test expression. This must be some expression that evaluates to a boolean value. This doesn't have to be enclosed in parentheses (as I've done in my examples below), but many programmers use parentheses to make the condition visually stand out. The condition is followed by a question mark (?), which separates it from the results portion.

The results portion is divided in half with the true portion (that is, the value to be used if the condition is true) appearing on the left and the false portion appearing on the right. The two results are separated by a colon. The expressions used for the true and false portion can any type at all, but both halves must be the same type, or at least assignment compatible types. You can't supply a String for the false portion and an int for the true portion as shown in the second line here. That just doesn't work.

You can also nest another conditional expression (using the conditional operator) inside the true or false portions of a first conditional expression. This quickly becomes unreadable, however, and, despite what you might think, may even execute more slowly. You should attempt to write source code that is clear and to-the-point, not code that uses the fewest possible lines.



Express Yourself

- **Exercise 3:** use Code Pad to complete the code shown below so that it's grammatically correct.

```
int quarters = (int)(Math.random() * 4);
System.out.printf("You have %d quarter%s%n",
    quarters,
    /* Your code goes here */);
```

- Use the selection operator
- If one quarter, use the empty string
- Otherwise, use the plural "s"

Notes:

Let's try an example using the selection operator. When writing programs that display output, you want them to sound grammatically correct. To do that, you have to handle plurals correctly. Let's see if you can do that with the selection operator and Code Pad.

Type the following code into Code Pad. The program creates an int variable named quarters and then initializes it to one of the values 0, 1, 2, or 3. You want to print out a message that tells the user how many quarters they have. The only problem is, you don't want to say: "You have 1 quarters", you want to

Slide 13

Express Yourself

Duration: 00:01:39

Advance mode: Auto

say "You have 1 quarter".

The `printf()` statement here uses three formatting placeholders. The `%d` will display the value stored in the variable `quarters`. The `%s` at the end of the word `quarter` is a String placeholder and it should display either "s" when you have zero or more than one quarters, or the empty string when there is one quarter. You'll initialize this placeholder by using the selection operator in the portion where it says "Your code here".

Run your code at least four times. (You can use the up-arrow in code pad so you don't have to type it in over and over). Each time, calculate a new value for `quarters` and then print it. (Don't just print the same value again and again.)

Shoot me two screen-shots; one of your Code Pad window and one of your output window similar to the one I've shown here.